PROJETO INTEGRADOR · UNIVESP

Monitor de Ruído IoT

Aplicativo Android para transformar dados de sensores em informação clara, contextualizada e acionável.

Equipe: preencher nomes · Banca: preencher data

01 · CONTEXTO

O problema

Dados brutos de sensores não respondem sozinhos à pergunta mais importante: “o que está acontecendo agora e isso exige atenção?”

Baixa interpretação

Leituras técnicas são difíceis de compreender por usuários não especialistas.

Falta de contexto

O mesmo nível pode ser permitido durante o dia e gerar alerta no horário de silêncio.

Rede instável

Uma falha temporária não deveria apagar a última informação confiável.

Decisão norteadora

Como monitorar em quase tempo

02 · OBJETIVO

Uma experiência simples para um problema técnico

Desenvolver um aplicativo Android que converta medições do ambiente em uma visão operacional das casas monitoradas.

01

Consultar a API do servidor local.

02

Interpretar os níveis em linguagem simples.

03

Manter o último estado durante falhas temporárias.

Critério de sucesso: o usuário identifica rapidamente casas barulhentas, sensores sem sinal e o contexto do horário de silêncio.

03 · ARQUITETURA

O caminho do dado

ESP32

Medição do ambiente

↓

Docker / FastAPI

Processamento, regras e API

↓

Flutter

Interface, interpretação e cache

Separação de responsabilidades: o servidor concentra a origem dos dados; o aplicativo concentra a experiência de consulta.

04 · FUNCIONAMENTO

Do sensor à tela

1. O sensor mede o ambiente e envia os dados pela rede.
2. O servidor recebe, processa e disponibiliza respostas JSON.
3. O app consulta o overview a cada 5 segundos.
4. O usuário visualiza estado, alertas e sensores offline.
5. O detalhe da casa consulta o histórico diário.
6. Respostas válidas são salvas localmente para contingência.

```
GET /api/app/overview
GET
/api/app/houses/{sensor_id}/histor
```

05 · IMPLEMENTAÇÃO

Arquitetura interna do app

COMPONENTE	RESPONSABILIDADE
main.dart	Inicialização e tela inicial
api.dart	Cliente HTTP, modelos e parsing defensivo
settings.dart	Endereço salvo e cache persistente
noise_ui.dart	Tema, cores e componentes reutilizáveis
screens/	Conexão, overview, detalhe e ajuda

06 · EXPERIÊNCIA

O usuário não precisa conhecer o protocolo

Conectar

Informar o IP do computador. A conexão é testada antes de ser salva.

Tranquilo

Movimento normal

Barulhento

Muito alto

Sem sinal

Entender

O app traduz níveis técnicos para rótulos e exemplos do cotidiano.

Agir

Alertas destacam ruído alto dentro do horário de silêncio.

Investigar

O usuário abre uma casa e consulta o histórico visual

07 · CONTEXTO

Horário de silêncio e alertas

O alerta não depende apenas da intensidade: ele depende da intensidade *e* do momento.

A

Período ativo

O servidor informa se o silêncio está vigente.

\+

Nível alto

Ruído alto ou muito alto é identificado.

!

Alerta

A combinação gera destaque visual para o usuário.

Decisão de produto: separar “sensor sem sinal” de “ambiente silencioso” evita uma interpretação incorreta.

08 · VISUALIZAÇÃO

Histórico que ajuda a explicar o dia

24
intervalos horários exibidos no gráfico.

RMS
média de intensidade usada para dimensionar as barras.

∞
pior nível registrado por horário.

3
resumos: duração, hora mais agitada e medições.

As faixas azuladas representam o horário de silêncio e conectam o dado técnico ao contexto do condomínio.

09 · CONFIABILIDADE

Quando a rede falha, a informação permanece útil

Resposta válida

Dados recebidos da API

↓

Cache local

Overview e histórico persistidos

↓

Falha temporária

Último estado + idade exibidos

Timeout

6 segundos para evitar espera indefinida.

Transparência

A interface informa quando os dados são antigos.

10 · QUALIDADE

Entrega acompanhada de validação

✓

Parsing

Tratamento defensivo para campos opcionais e tipos inesperados.

✓

Testes

Normalização de URL, parsing e widgets principais.

✓

CI/CD

Analyze, test, build e upload do APK automatizados.

Pipeline: dependências → análise → testes → build release → artefato.

11 · SEGURANÇA

Escopo claro e evolução responsável

O que o app não faz

Não grava áudio, não controla sensores e não altera medições.

Limitação atual

HTTP sem TLS é usado apenas para a rede local confiável.

Antes da exposição externa

Adicionar HTTPS, autenticação, autorização e revisão de segurança.

Precisão

Os níveis são classificações operacionais, não medição legal em decibéis.

12 · RESULTADO

O que foi entregue

- Aplicativo Android organizado em Flutter.
- Integração definida com a API do servidor.
- Visão geral, alertas, sensores offline e histórico.
- Cache persistente para falhas temporárias de rede.
- Testes, análise estática e build automatizado.
- Documentação operacional para execução e manutenção.

Valor entregue: reduzir a distância entre a medição técnica e a decisão do usuário.

13 · ADMINISTRAÇÃO

Moradores e notificações

Configuração local

O protótipo permite cadastrar nome, contato, casa/sensor e canal preferido.

Produção

O administrador deverá usar login, papéis, convites e endpoints protegidos no backend.

Push

O backend registra dispositivos e usa Firebase Cloud Messaging para alertas com o app fechado.

E-mail

O e-mail funciona como canal complementar;

Ponto pendente: o backend FastAPI/Docker e as credenciais de produção não fazem parte deste repositório.

14 · EVOLUÇÃO

Próximos passos

Integração

Versionar formalmente o contrato da API e testar com servidor simulado.

Validação

Executar testes em dispositivo real com quatro sensores.

Produto

Melhorar acessibilidade, internacionalização e observabilidade.

Segurança

Adotar HTTPS e autenticação para uso fora da rede local.

CONCLUSÃO

Dados claros. Decisões melhores.

O Monitor de Ruído IoT transforma medições do ambiente em uma experiência visual, contextualizada e resiliente.

Obrigado · Perguntas?